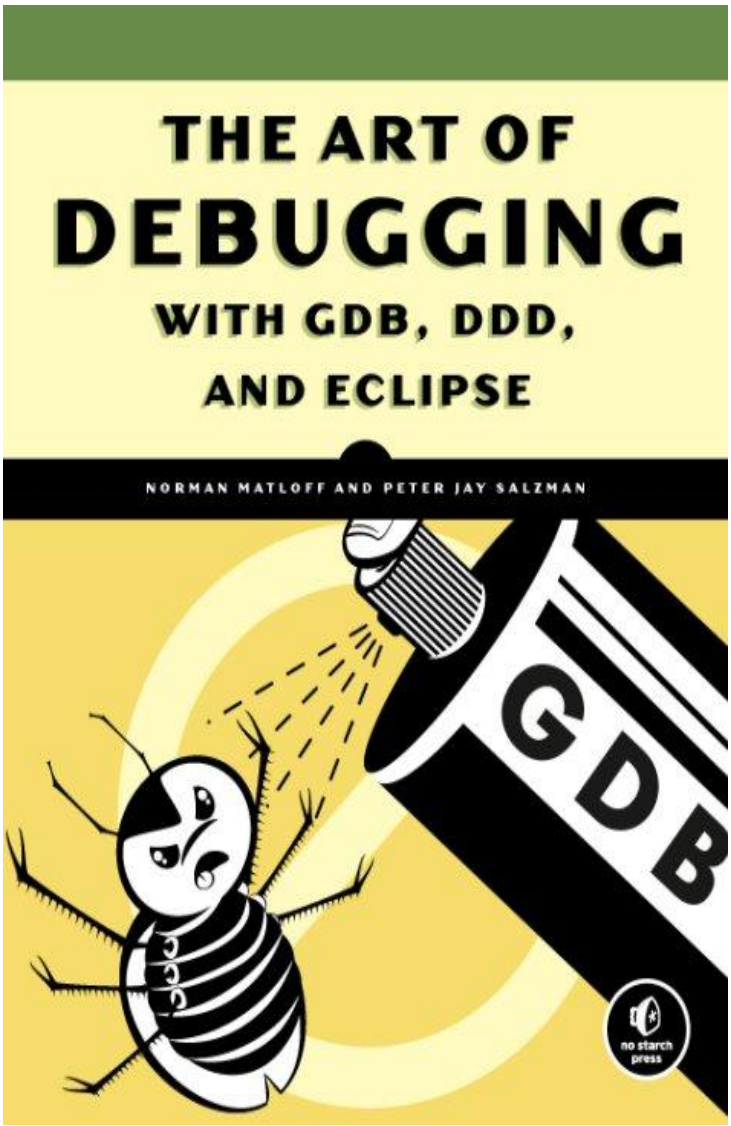
DEPURACION DE CÓDIGO CON ECLIPSE



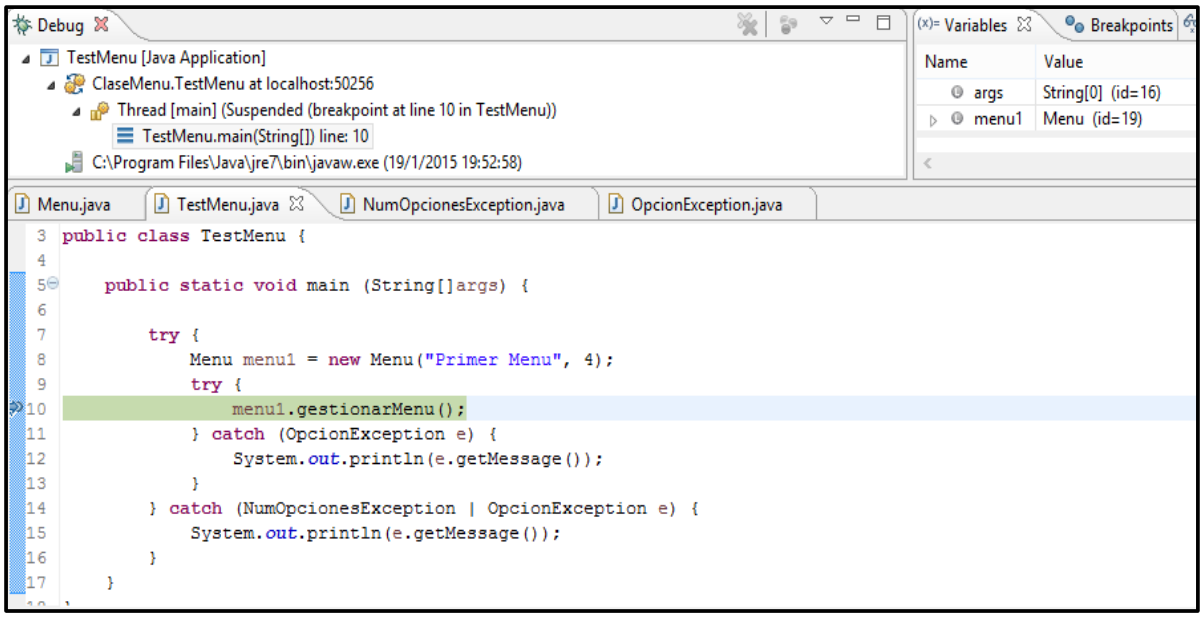
Rafael Miranda Ibáñez

Emanuel Galvan Fontalva

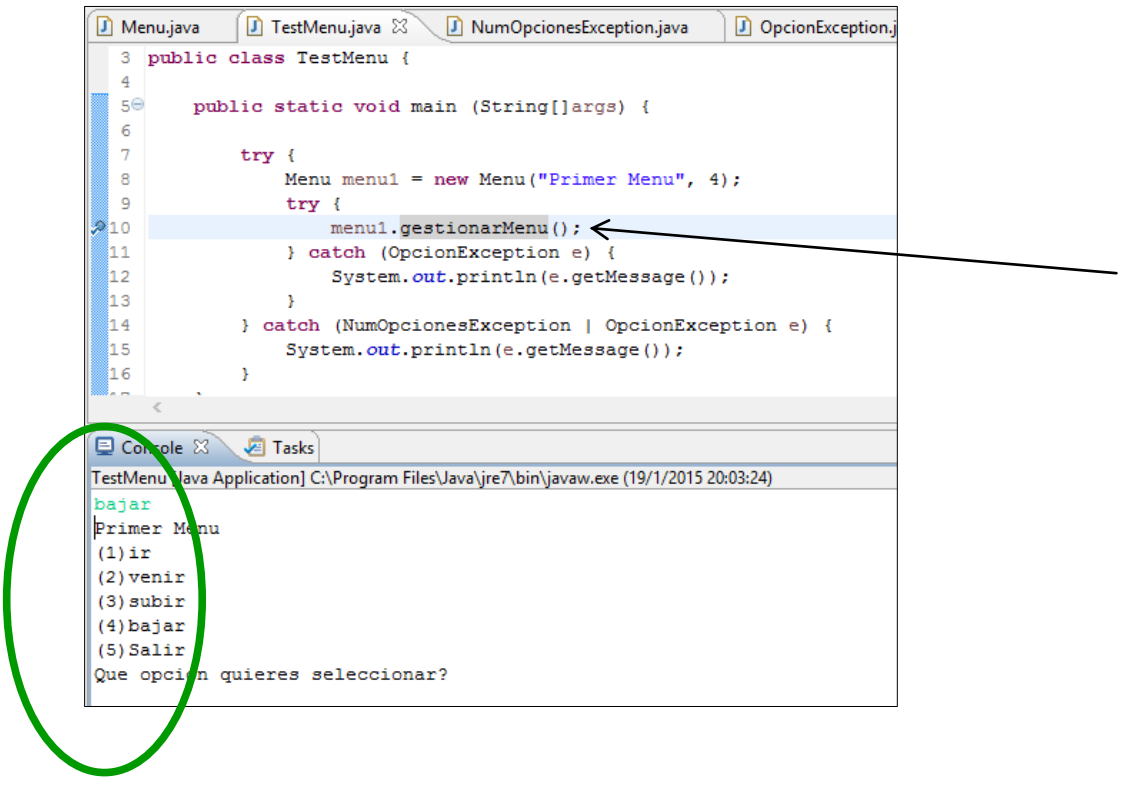
COMANDOS STEP OVER, STEP INTO, STEP RETURN Y RESUME

Step Over (F6) nos permite ir ejecutando linea a linea un código en concreto sin que haga salto para entrar en la ejecución linea a linea del método al que ese código pueda invocar. Es decir nos muestra el resultado en consola, sin ninguna alteración, pero no nos da el detalle de la ejecución linea a linea del método invocado.

Ejemplo: Usamos como código de muestra nuestro paquete ClaseMenu, y en TestMenu colocamos un breakpoint en la línea que invoca al método "gestionarMenu()".



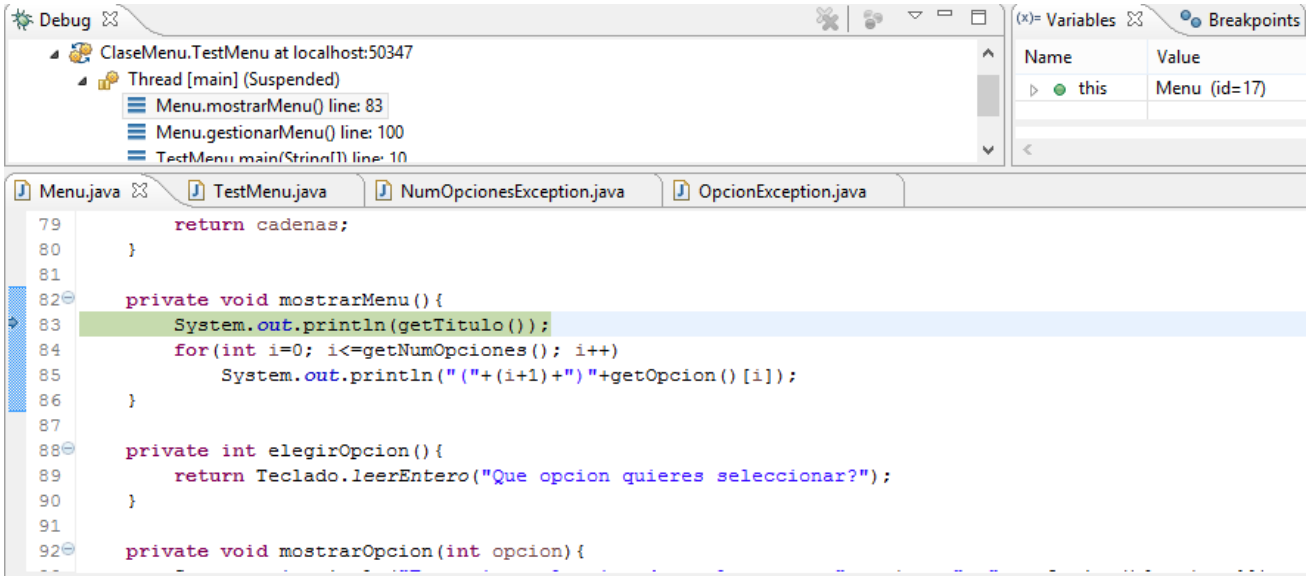
Tras pulsar F6 o el icono Step Over observamos que el área del editor de código no salta a la clase "Menu" donde está declarado ese método y no nos muestra su ejecución línea a linea sino que nos muestra la salida en pantalla sin más.

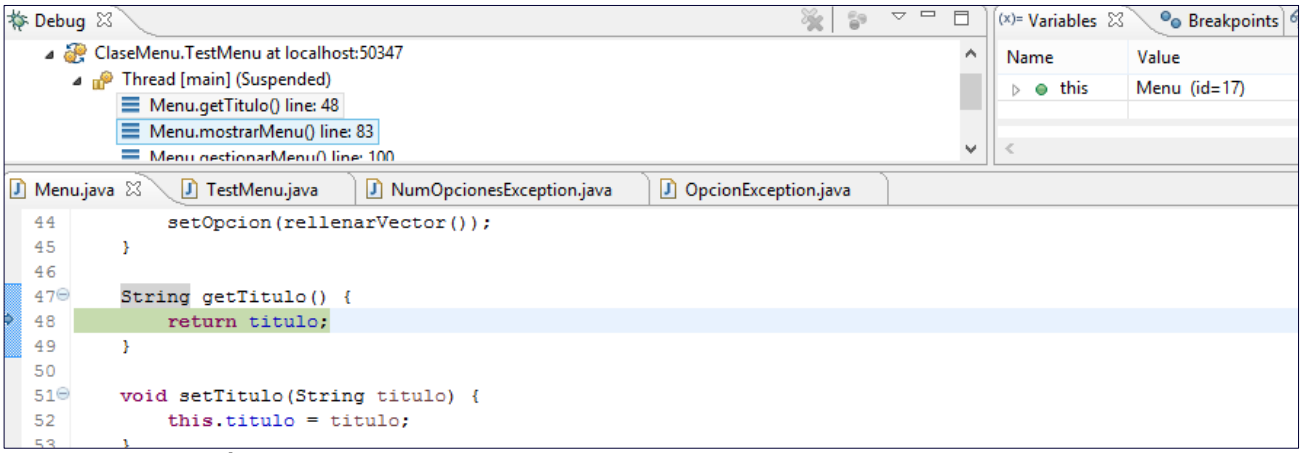


Step Into (F5), al contrario que Step Over, si que hace el salto en la ejecución y nos muestra linea a linea la ejecución del código o método invocado. Cuando termina de ejecutarlo vuelve al código original que invocó ese método para continuar con su ejecución secuencial.

Para ejemplificarlo colocamos de la misma manera el breakpoint en la línea que invoca al método "gestionarMenu()". Pero ahora tras F5 o el icono Step Into observamos que el área del editor se desplaza a la clase "Menu", donde está declarado ese método (concretamente a la linea 100 donde se encuentra con el siguiente método) y muestra su ejecución línea a linea.

Como se observa en la siguiente imagen la ejecución salta al método "mostrarMenu()" y como su primera línea invoca a otro método ("getTitulo()") salta a la ejecución de





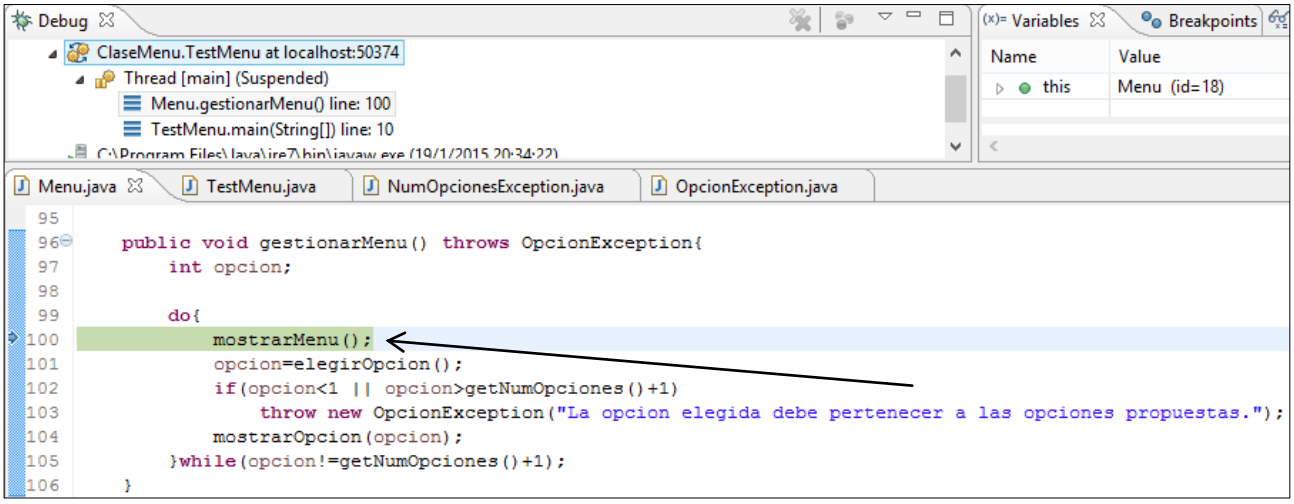
ese segundo método.

Step Return (F7) sirve para, una vez con Step Into estemos dentro de la ejecución de un método, poder salir de él en cualquier momento aunque no se haya terminado de ejecutar y volver de esta manera al código que lo invocó originalmente.

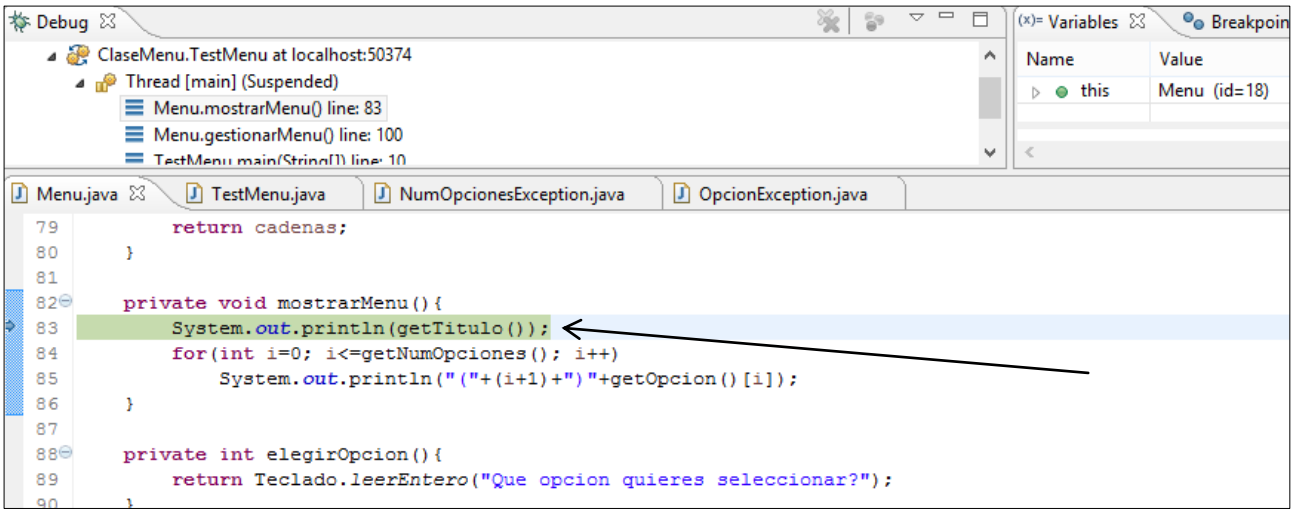
Para explicarlo volvemos al punto de partida, el breakpoint colocado en la invocación del método "gestionarMenu()".



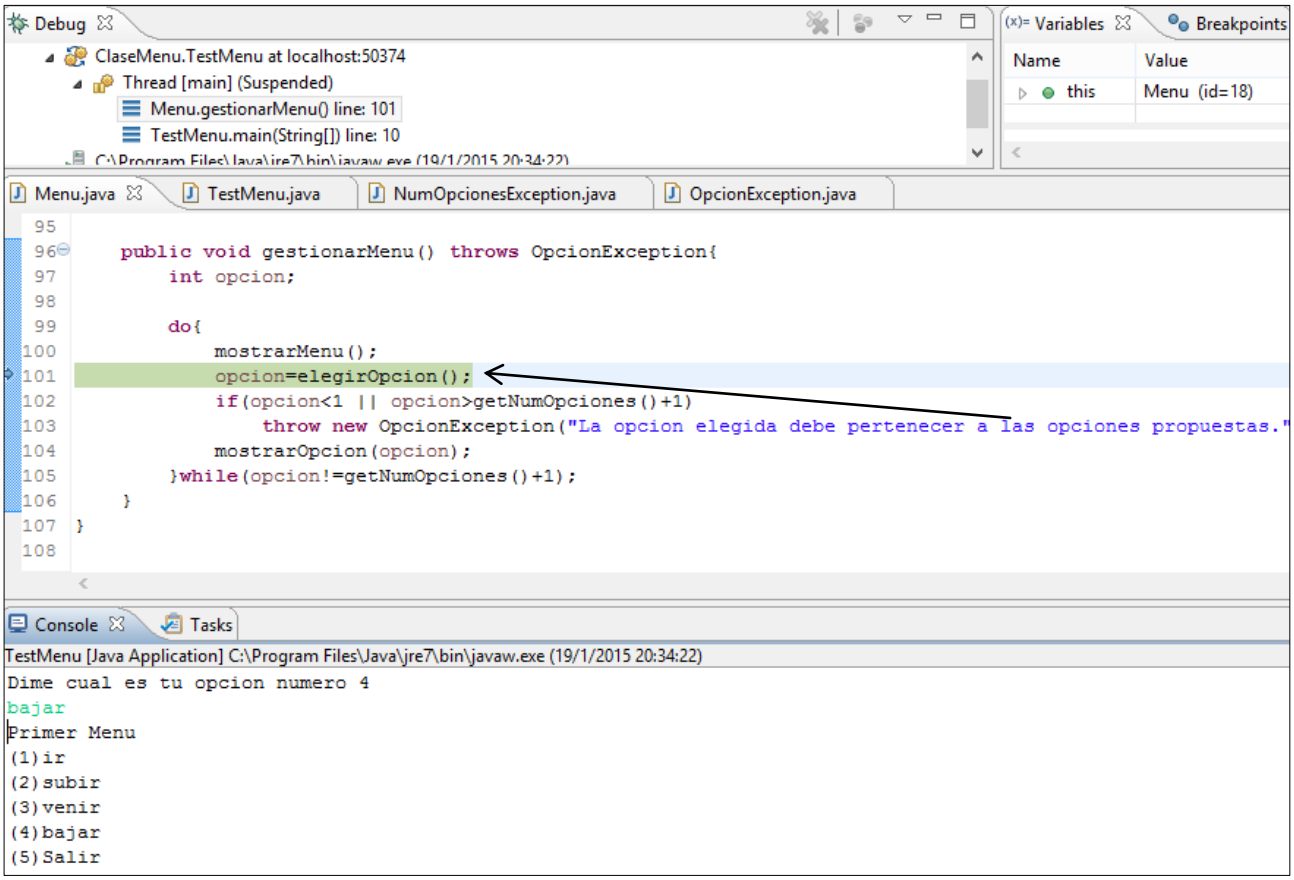
Si tras esto pulsamos Step Into el comportamiento será según lo explicado anteriormente, es decir, el área del editor se desplaza a la clase "Menu", donde está declarado ese método.



Si tras esto clicamos dos veces en Step Into continuará con el proceso descrito para este tipo de función.



Pero si tras esto pulsamos Step Return observamos como el editor del texto vuelve a la linea posterior al método "mostrarMenu()", que era dentro del cuyo flujo de ejecución estábamos, es decir, rompe con la pila de ejecuciones de métodos invocados a partir de "mostrarMenu()", para continuar con la siguiente línea habiendo sacado por



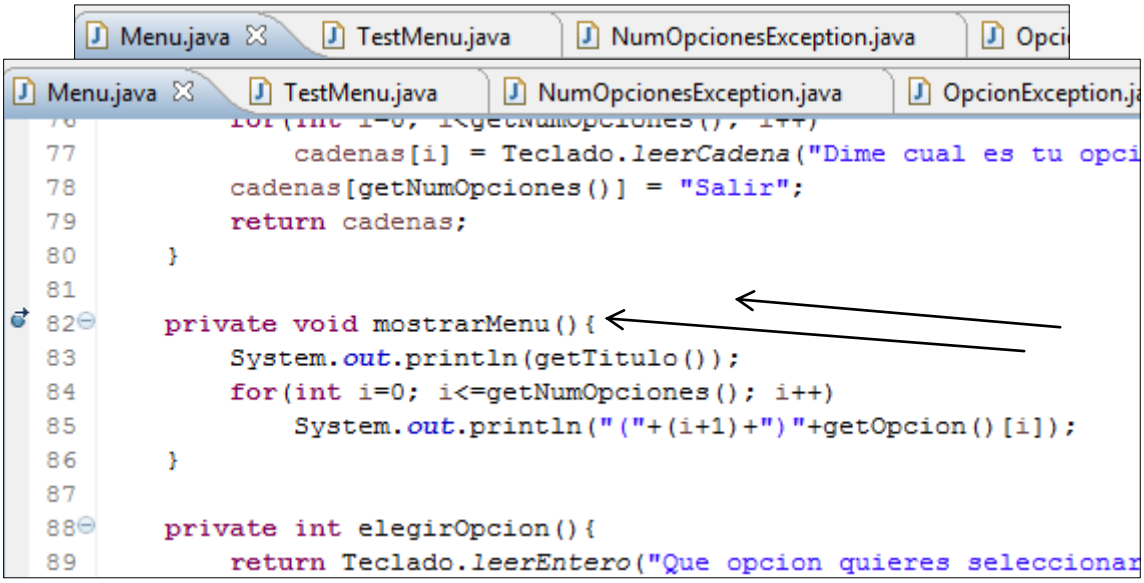
consola el resultado de "mostrarMenu()".

Como se observa en la pila de ejecuciones, tras pulsar Step Return automáticamente se elimina la pila superior y baja a la inferior.

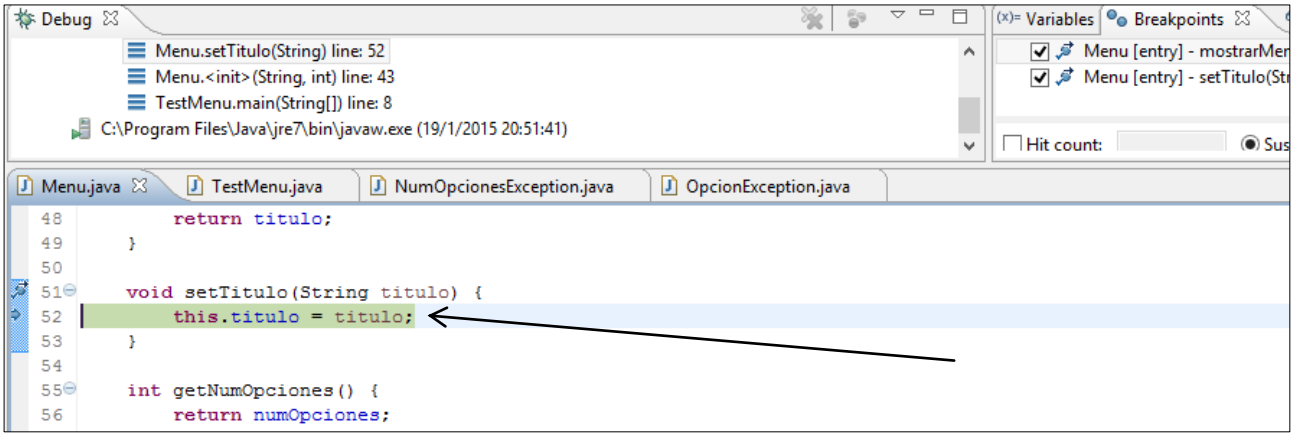
Y por ultimo **Resume (F8)** nos permite ejecutar el programa hasta el siguiente breakpoint o watchpoint (o hasta que finalice el programa) sin ver todas las instrucciones que se ejecutan hasta llegar a ese punto. Coloquialmente podríamos equipararlo a un atajo.

Para ello seleccionamos dos breakpoint, uno en "setTitulo()" y otro en la invocación del método "mostrarMenu()".

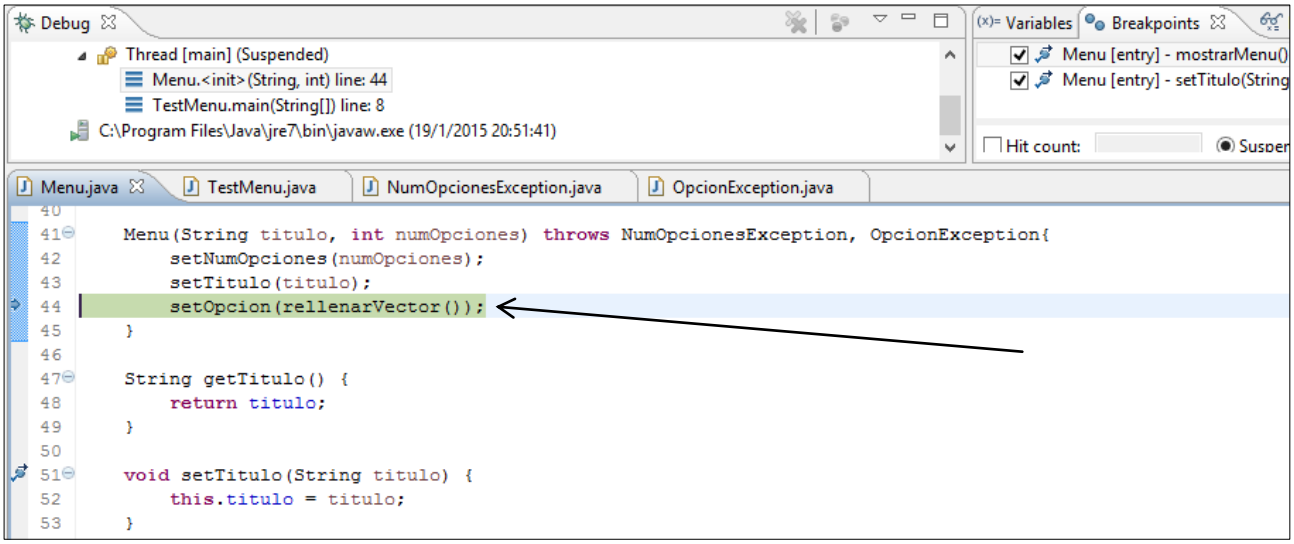
Si



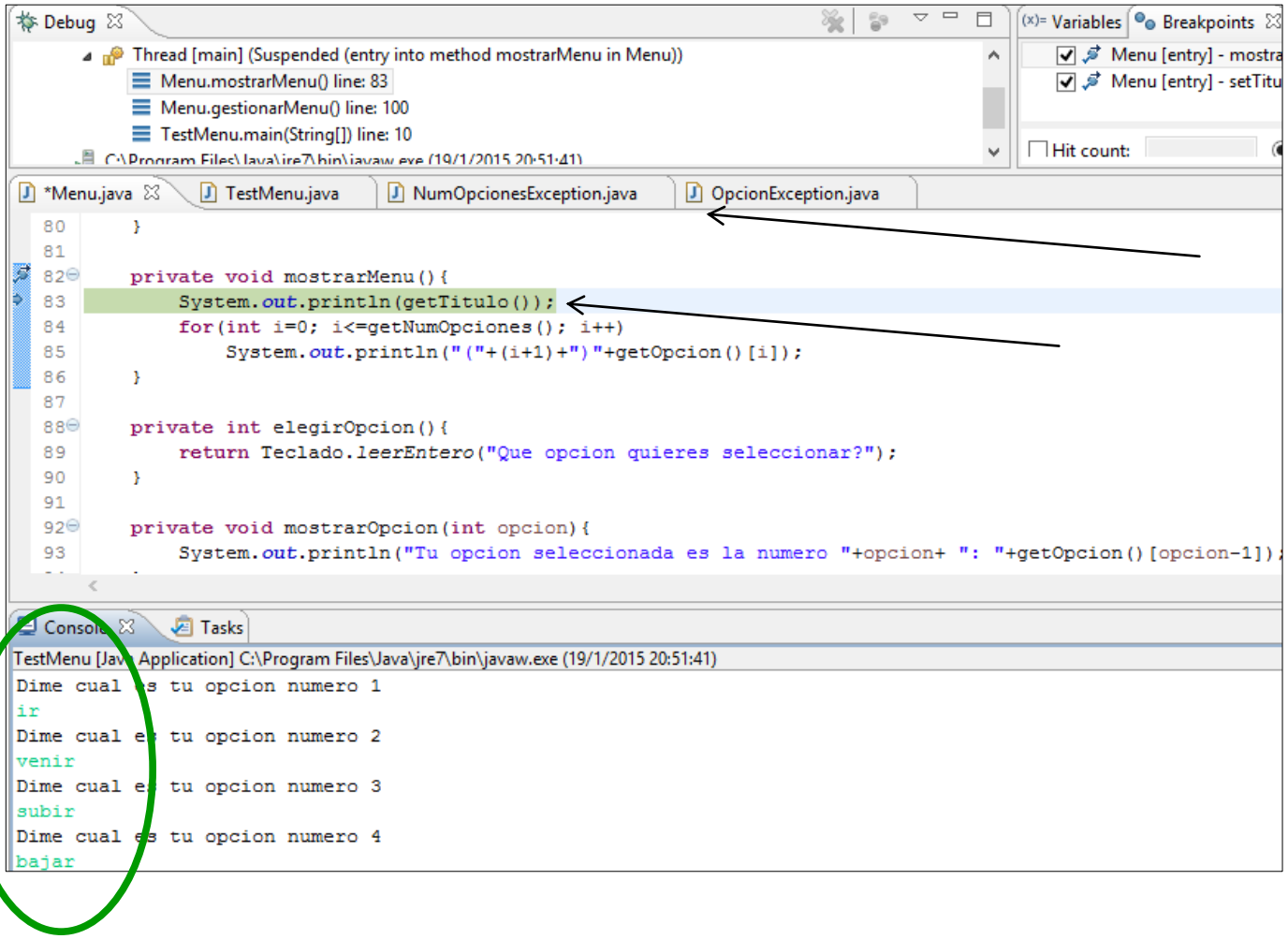
pulsamos Step Into comienza la ejecución linea a linea del código, en este caso dentro del método "setTitulo()".



Si continuamos clicando el comportamiento será el descrito, ejecución linea a linea, y por tanto tras ejecutar "setTitulo()" se desplaza al siguiente setter del constructor.



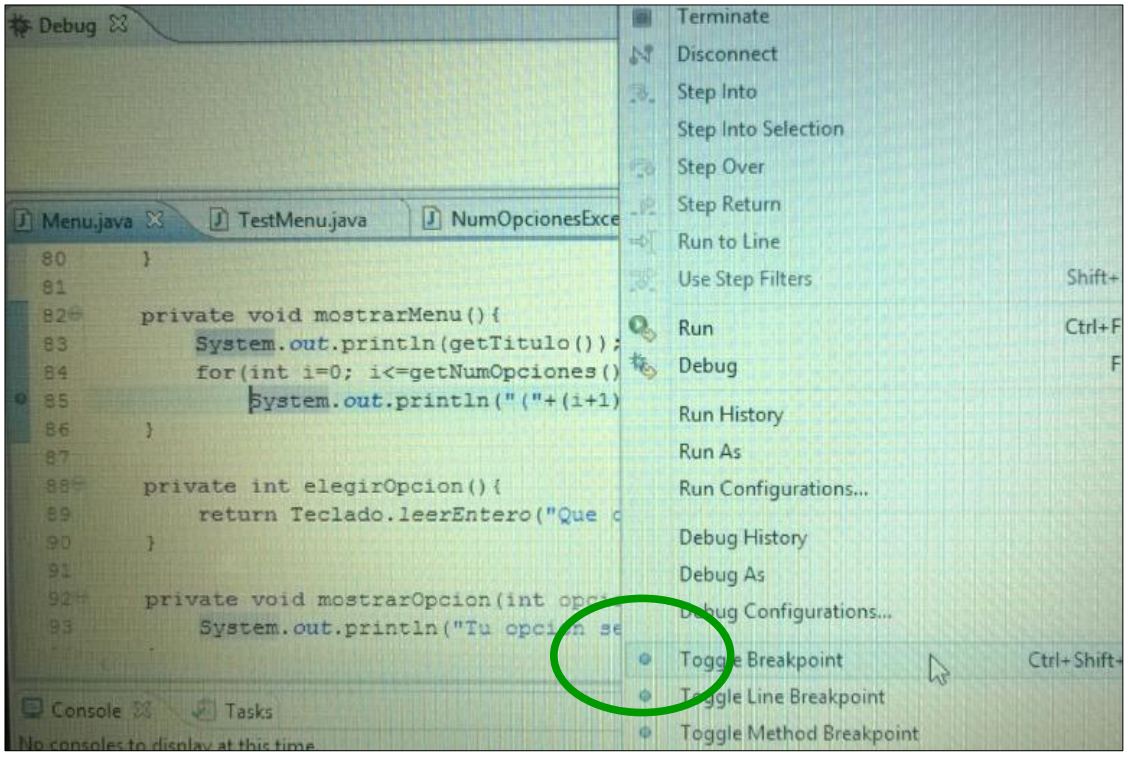
Pero si ahora pulsamos Resume (F8), irá directamente a la línea del siguiente breakpoint seleccionado ("mostrarMenu()") o en su defecto finalizará el programa.



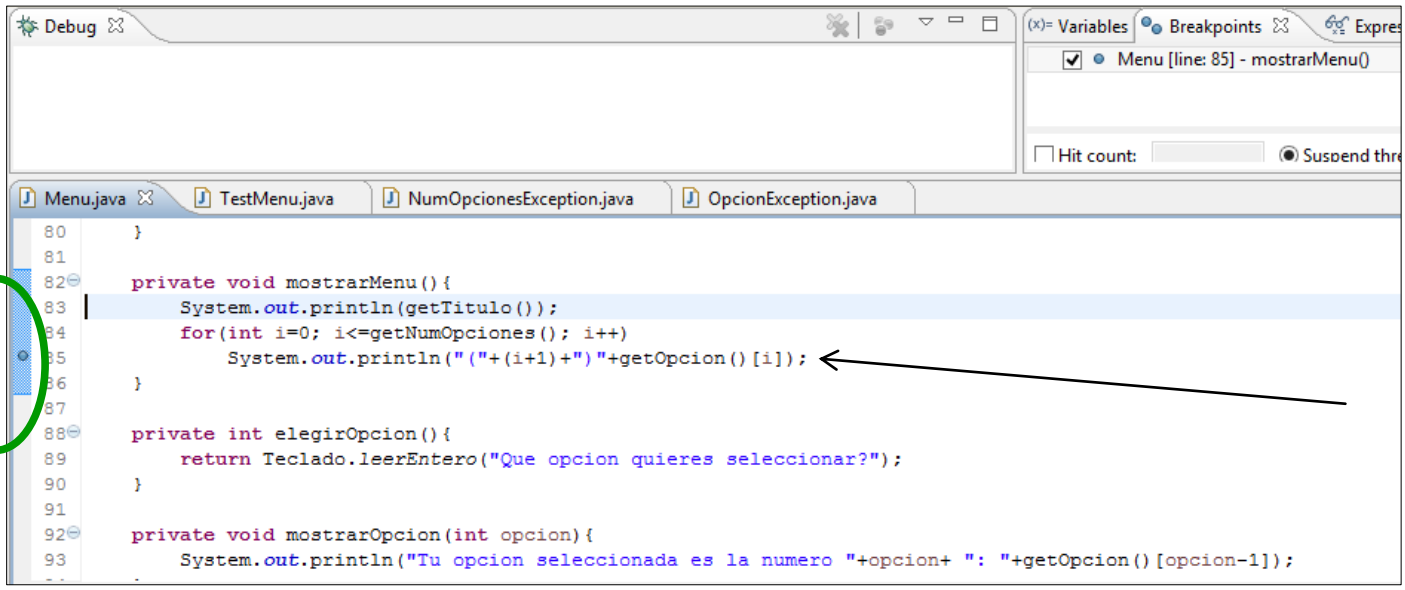
DIFERENCIAS ENTRE BREAKPOINT, WATCHPOINT Y PUNTOS DE RUPTURA DE EXCEPCION

Los puntos de ruptura (breakpoint) los situamos en aquellas linea del código donde queremos que la ejecución se detenga para poder evaluar si el comportamiento del programa es el que pretendíamos con nuestro diseño. Para definir un punto de ruptura (o para eliminar uno existente), nos situamos en la linea que queramos y seleccionamos "Run -> Toggle breakpoint".

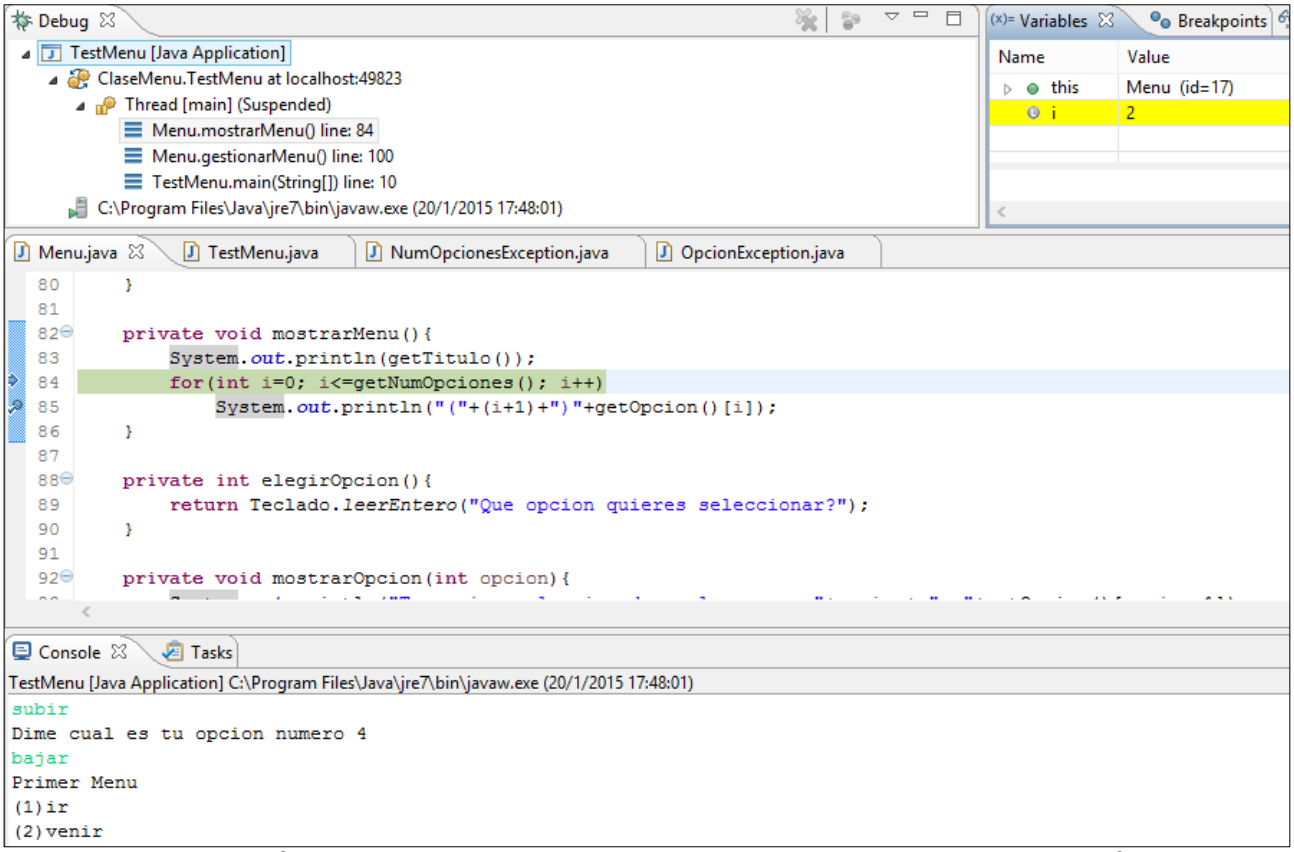
o



hacemos doble-click en la barra a la izquierda de la ventana del código. Al definir un punto de ruptura aparecerá en dicha barra un punto color azul.



Así como ya hemos visto en el apartado anterior del tutorial mediante este punto de ruptura se establece la linea de codigo donde la ejecución debe detenerse para

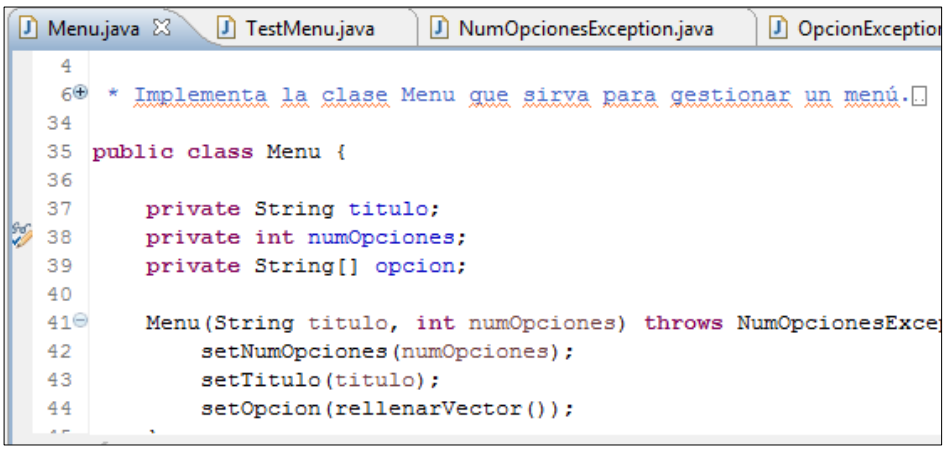


estudiar la evolución del programa, valor de las diferentes variables y demás.

Así, en este ejemplo se observa como la zona de información de variables y breakpoints nos informa que la variable "i" en esta iteración del "for" ha conseguido su valor 2, por lo que ya muestra en pantalla la segunda posible opción del menu.

Esta denominación, breakpoint, es la genérica para referirnos a esas líneas donde queremos observar el comportamiento. Pero existen variantes para líneas de código concretas como el watchpoint que hace alusión al mismo concepto pero para líneas de

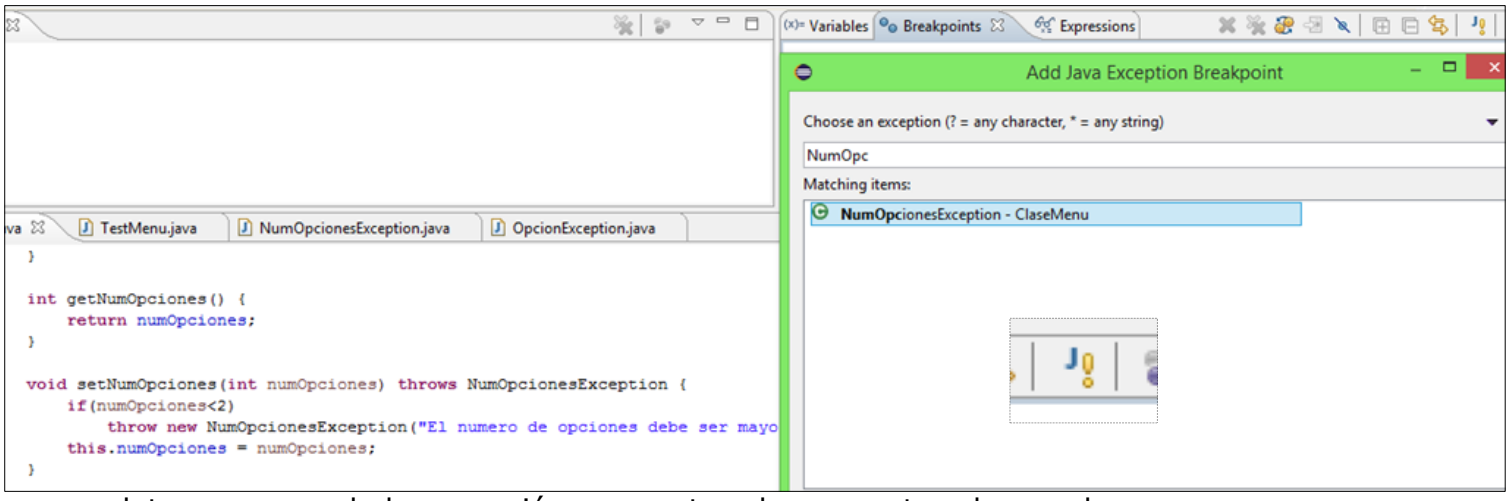
código que contienen declaraciones de campos dentro de una clase determinada. Para crear un watchpoint se procede de la misma manera que con los breakpoints, es decir, hacemos doble click en la barra de la izquierda a la misma altura que la linea de código que contiene el campo. En sus propiedades podemos configurar si queremos que se detenga la ejecución cuando se lea el campo, cuando se modifique o ambas.





El funcionamiento es la misma que la descrita para los breakpoints de cualquier línea de código.

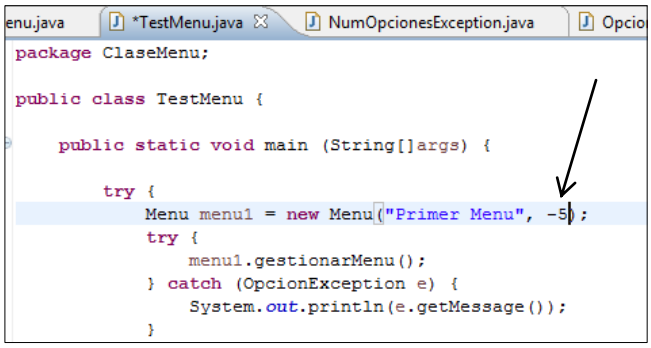
Los breakpoint de excepciones detienen la ejecución cuando salta una excepción del tipo seleccionado. Estos puntos de ruptura se activan haciendo clic en el icono "J!" de la vista de "Breakpoints" o desde el menú principal "Run". La ejecución puede



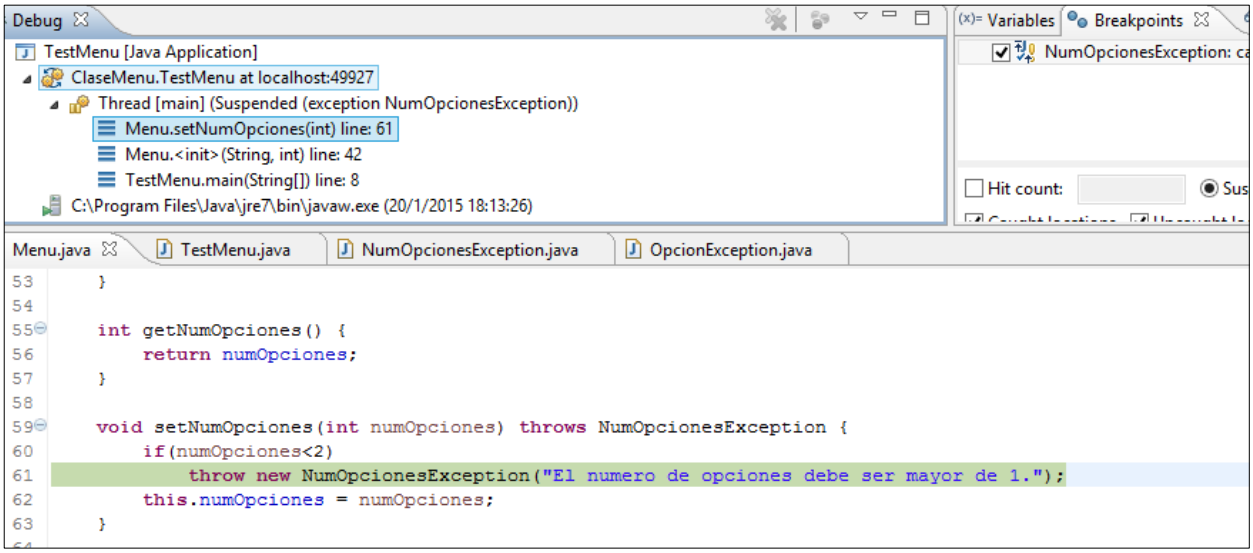
detenerse cuando la excepción sea capturada, no capturada o ambas.

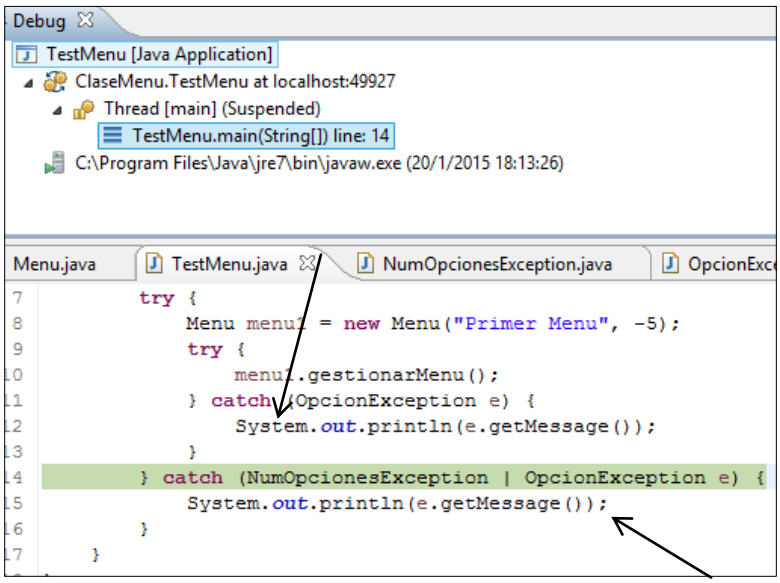
Como vemos en la imagen anterior seleccionamos la excepción que queremos monitorizar escribiendo su nombre en el menú que incluye todas las existentes tras pulsar el icono.

Como la excepción trata los casos en los que el numero de opciones introducidas no es válido (como es el caso de los valores inferiores a 2 porque no tiene sentido un menu de una opción o numeros negativos), creamos nuestro objeto menu con un valor de numero de opciones.

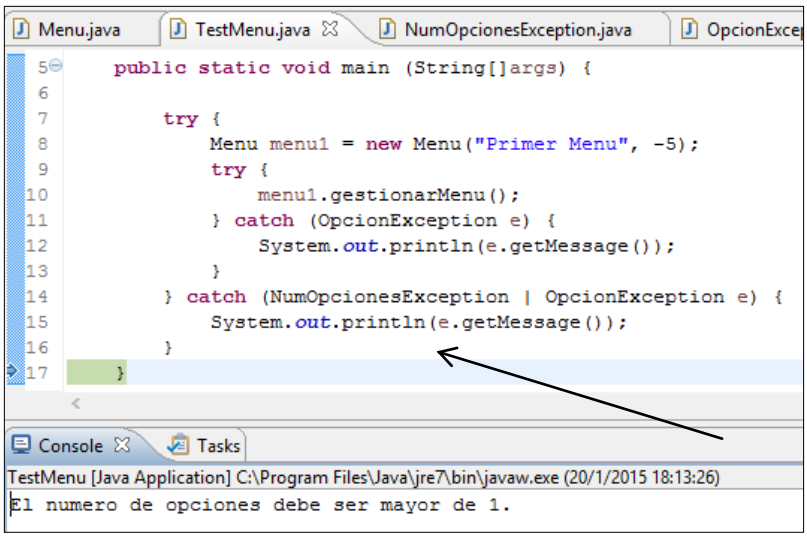


Al lanzar el depurador se detiene en la linea que lanza la excepción seleccionada





Como vemos en la anterior imagen la ejecución salta automáticamente a la zona try-catch y más concretamente al catch ya que no ha podido ejecutar la construcción del nuevo objeto menu1 ya que no cumple la restricción que dice que el valor debe ser mayor de 1.



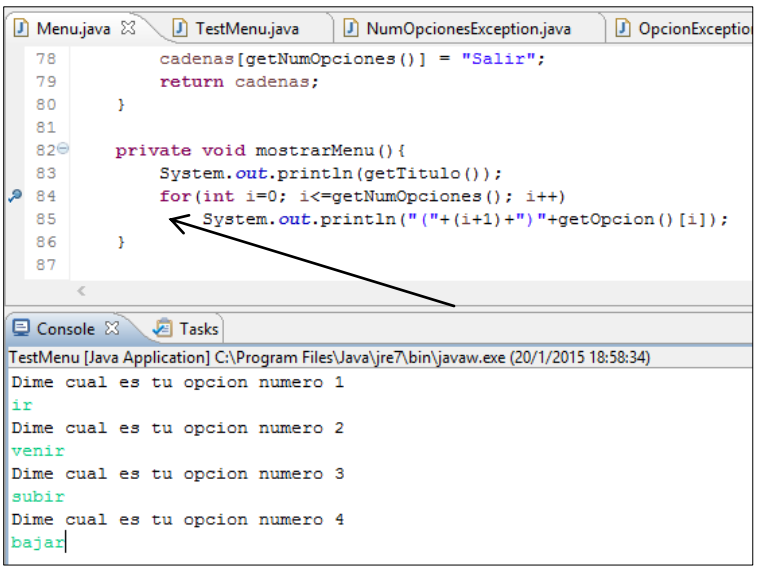


Añadir siempre los puntos de ruptura de excepciones Java de "ArrayIndexOutOfBoundsException" (lanzada cuando el índice de una matriz se sale de sus dimensiones) y "NullPointerException" (lanzada cuando se intenta acceder a una referencia que apunta a null) es una práctica de depuración recomendada.

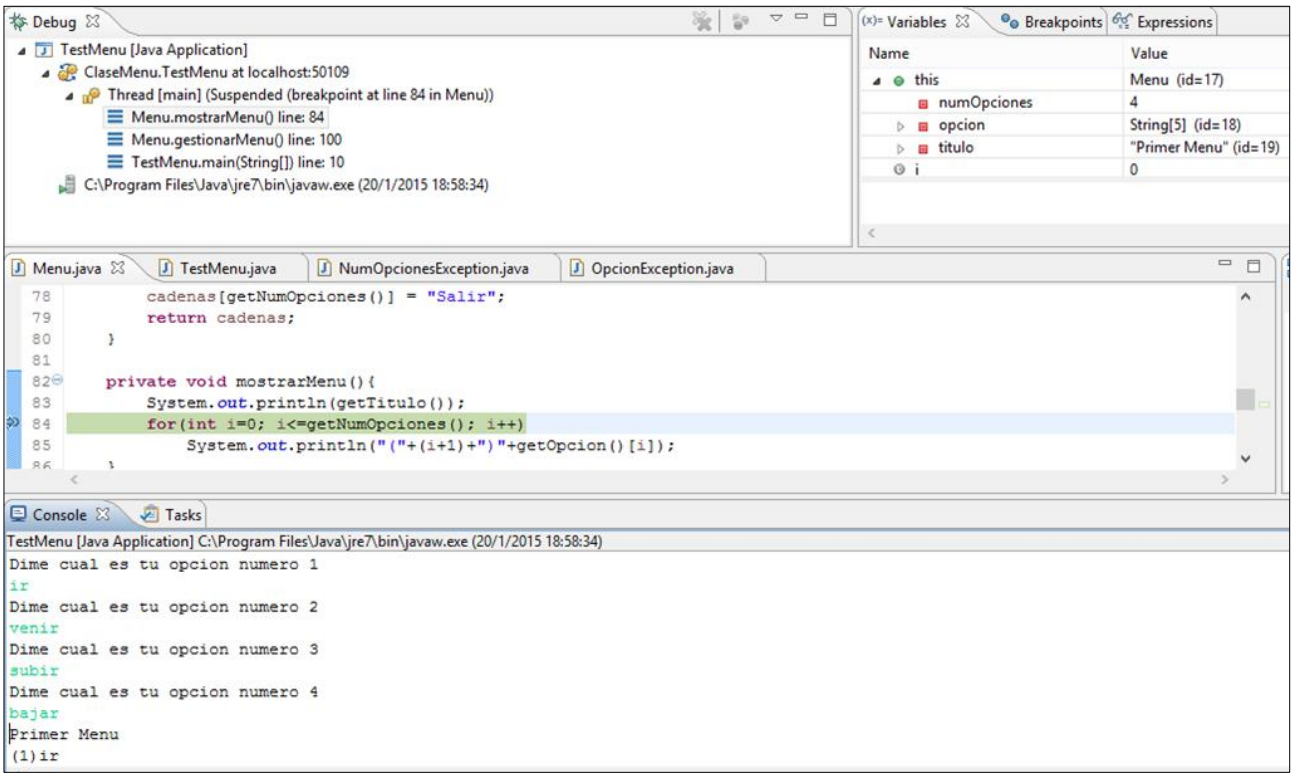
CAMBIO DEL VALOR DE VARIABLES EN UNA SESIÓN DE DEPURACIÓN

El depurador de Eclipse nos permite obtener información acerca de que valor van tomando las diferentes variables que intervienen en nuestro programa de manera que incluso podemos modificar ese valor para, artificialmente, deducir si el funcionamiento está siendo el deseado.

Usaremos como ejemplo un recorrido de un "for" que muestra las diferentes opciones que un usuario ha introducido como su menu. Para ello colocamos el breakpoint en la linea que contiene el inicio de ese "for". Tras esto lanzamos la ejecucion y el primer paso es pedirle al usuario cuales son sus cuatro opciones para el menu que quiere crear. Introducimos "ir", "venir", "subir", "bajar".

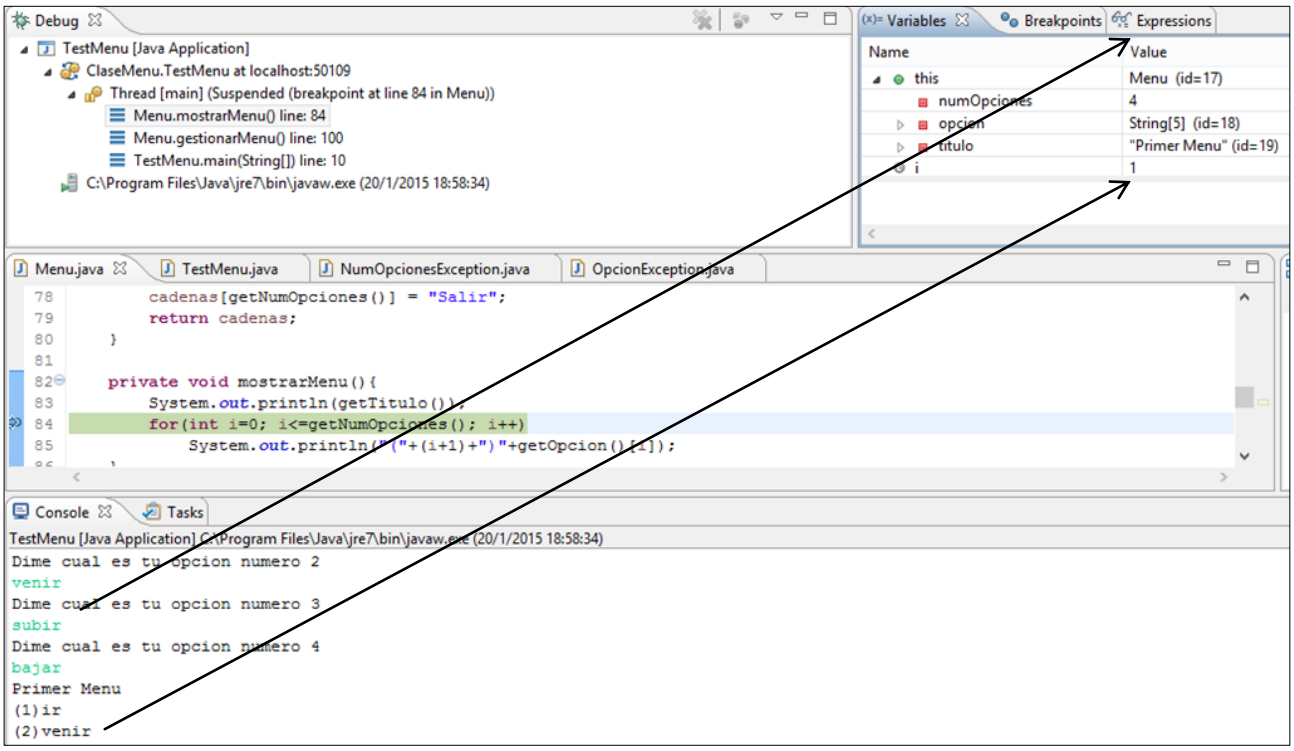


Vemos como, pulsando Step Over se va haciendo el recorrido y la variable "i" va incrementandose además de ir apareciendo en pantalla las opciones correspondientes

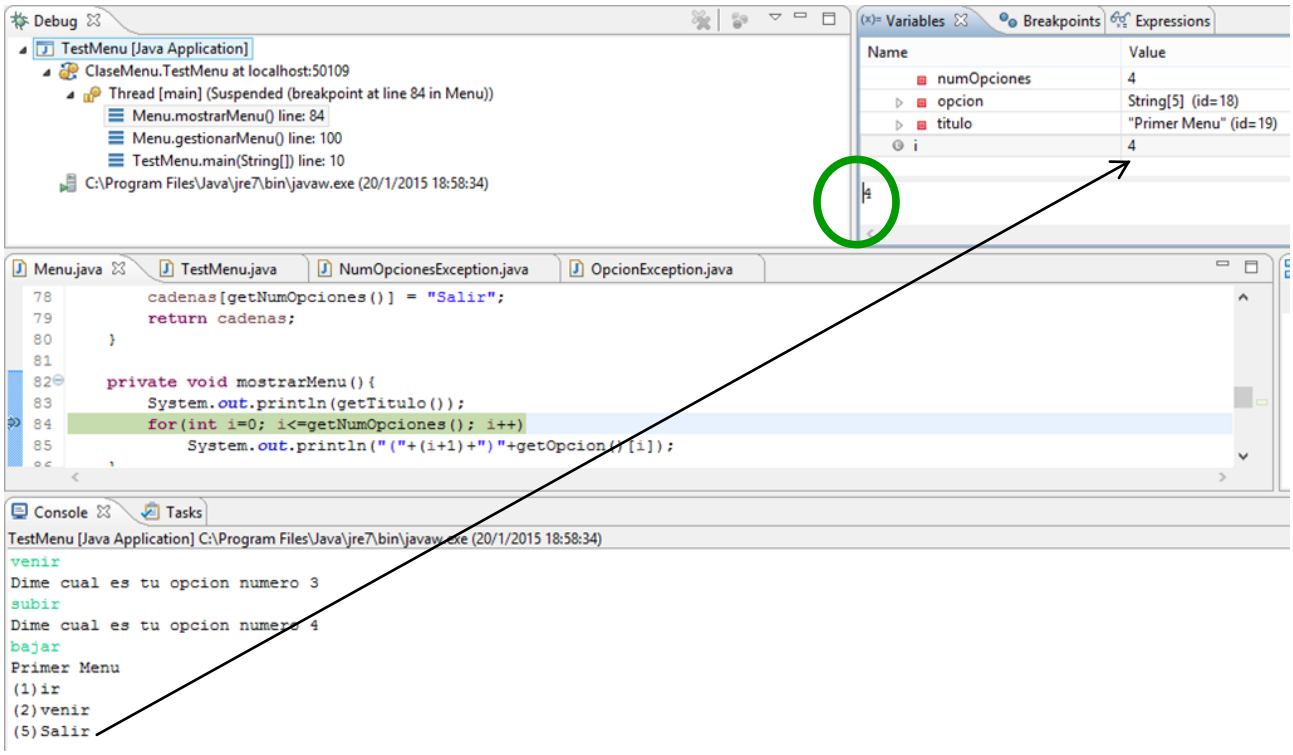


a cada “i”.

Se empieza por el valor 0 de “i”, correspondiente a la opción “ir”. Si volvemos a pulsar Step Over aparece el valor 1 de “i” y su opción “venir” del menu.



Y si ahora modifico manualmente el valor de la siguiente “i” y le pongo el valor 4 veremos como por pantalla, en lugar de aparecer la tercera opción, que recordemos es “subir”, aparecerá la ultima opción (salir), correspondiente con el valor 4 de “i”.

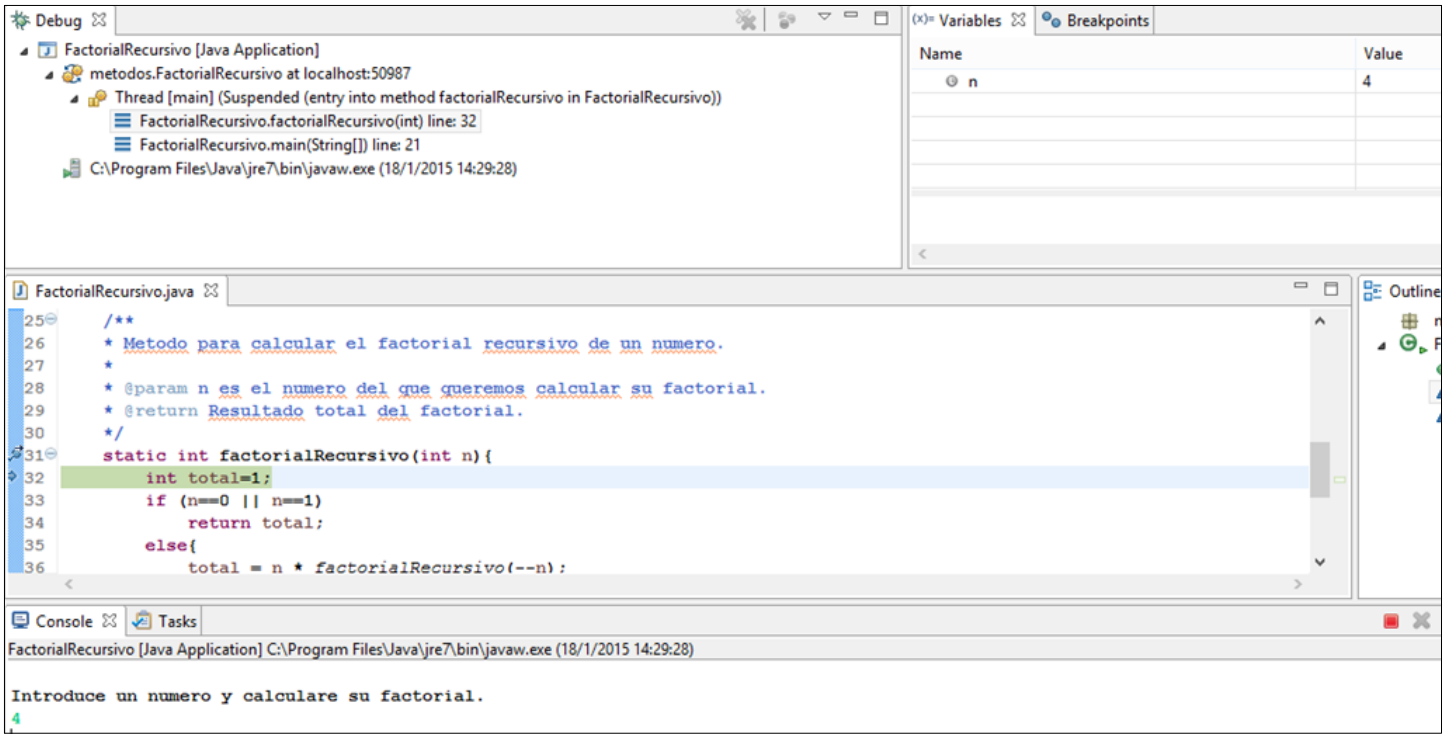


APLICACIÓN DEL DEPURADOR EN UN MÉTODO RECURSIVO

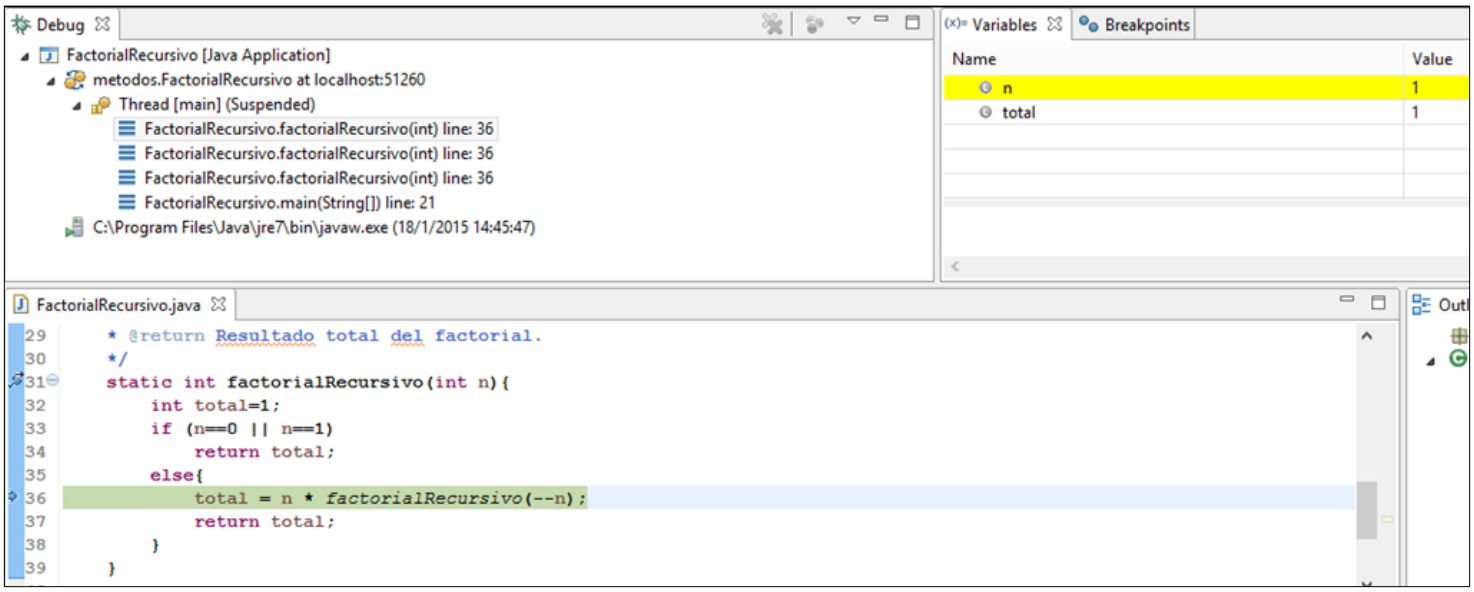
Código de partida:

```
static int factorialRecursivo(int n){
    int total=1;
    if (n==0 || n==1)
        return total;
    else{
        total = n * factorialRecursivo(--n);
        return total;
    }
}
```

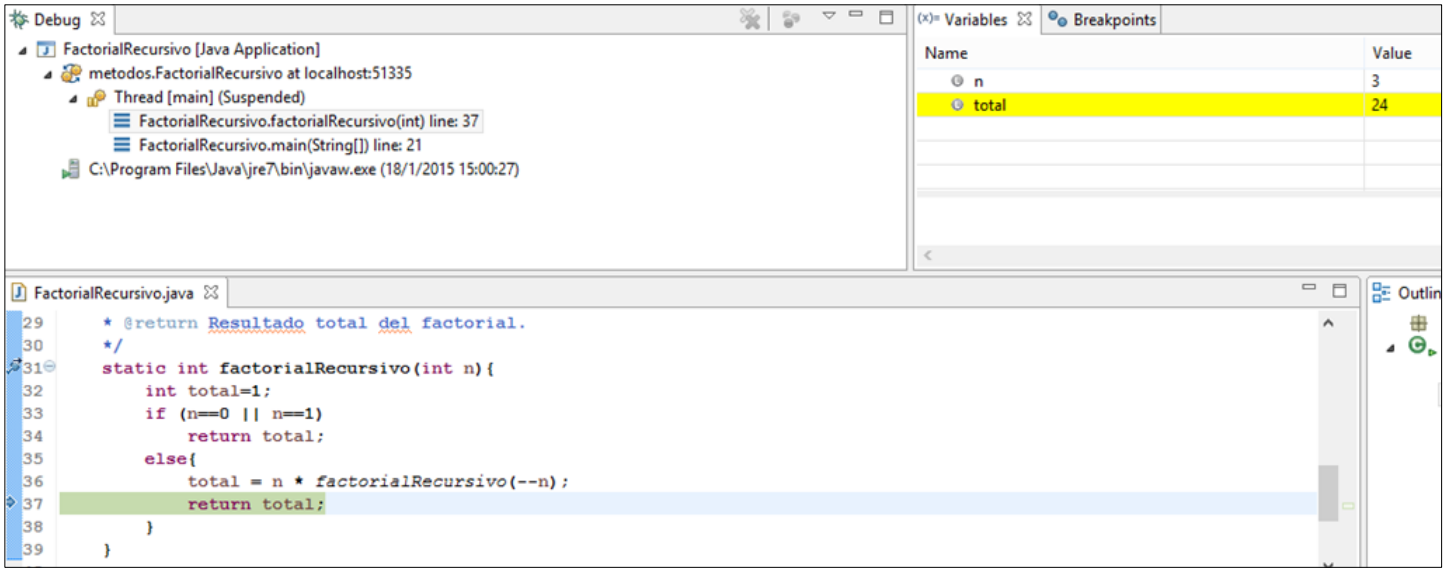
Cuando introducimos el numero del que queremos calcular el factorial (que nos lo pide anteriormente en el Main), el programa va directamente al método que calcula el resultado, y donde hemos colocado el breakpoint.



Así, cada vez que pulsamos en Step Over o Step Into comprueba la condición inicial y al entrar en else vuelve a invocar al método con lo que hace esa iteración tantas veces como resulte el numero introducido menos uno (porque factorial de uno es la unidad). En este caso como nuestro numero introducido es cuatro hace la iteración tres veces.



Posteriormente entra ya a ejecutar el producto y la suma del factorial recursivo y en la ventana del margen derecho (variables) se podrá observar como decrementa el valor de "n" desde 4 a 1 y el total desde 1 a 24 que es resultado de factorial de 4.



El código que vamos a usar para entender los bucles for y for mejorado es el siguiente:

```java
/**
 * Leyendo un vector de enteros con un for mejorado.
 *
 * @author Emanuel Galván Fontalba
 * @version 1.0
 */

public class LeyendoVectores {

    public static void main(String [] args){
        int[][] vector= {{0,12,15},{251,5,2},{42,75,452}};

        System.out.println("For Mejorado: ");
        muestraVectorForMejorado(vector);
        System.out.println("\nFor Clasico: ");
        muestraVectorForClasico(vector);

    }

    public static void muestraVectorForMejorado(int[][] vector){
        for (int i[] : vector) {
            System.out.println();
            for(int x: i)
                System.out.print("\t"+x);
        }
    }

    public static void muestraVectorForClasico(int[][] vector){
        for(int i=0; i<vector.length; i++) {
            System.out.println();
            for(int j=0; j<vector[i].length; j++)
                System.out.print("\t"+vector[i][j]);
        }
    }
}

```

Ahora vamos a recorrer un bucle **for clásico** con el depurador de eclipse y así poderlo entender mejor:
Lo primero que vamos a hacer es poner un punto de ruptura en la linea del for, para que el depurador se nos pare ahí para estudiarlo linea a linea.

```java
    public static void muestraVectorForClasico(int[][] vector){
        for(int i=0; i<vector.length; i++) {
            System.out.println();
            for(int j=0; j<vector[i].length; j++)
                System.out.print("\t"+vector[i][j]);
        }
    }
}
```

Una vez establecido el punto de ruptura vamos a ejecutar en modo depurador y se nos parará justo en la primera linea del bucle.

```
27
28    public static void muestraVectorForClasico(int[][] vector){
29        for(int i=0; i<vector.length; i++) {
30            System.out.println();
31            for(int j=0; j<vector[i].length; j++)
32                System.out.print("\t"+vector[i][j]);
33        }
34    }
35 }
36
```

(x)= Variables Breakpoints

Name	Value
vector	(id=16)

Ya que se nos a parado en la primera linea del bucle, va a empezar a entrar y a recorrerlo(haciendo click en Step Over o bien F6).

```
28    public static void muestraVectorForClasico(int[][] vector){
29        for(int i=0; i<vector.length; i++) {
30            System.out.println();
31            for(int j=0; j<vector[i].length; j++)
32                System.out.print("\t"+vector[i][j]);
33        }
34    }
```

(x)= Variables Breakpoints

Name	Value
vector	(id=16)
i	0

Ya vemos que el valor de i se encuentra a 0, avancemos al siguiente.

```
28    public static void muestraVectorForClasico(int[][] vector){
29        for(int i=0; i<vector.length; i++) {
30            System.out.println();
31            for(int j=0; j<vector[i].length; j++)
32                System.out.print("\t"+vector[i][j]);
33        }
34    }
35 }
```

(x)= Variables Breakpoints

Name	Value
vector	(id=16)
i	0
j	0

Ya vemos que entra en el bucle de la j y toma el valor cero, vamos a avanzar otro bucle más.
Ahora nos imprime el valor:

```
For Clasico:

        0
```

Avanzamos y se vuelve a meter en el bucle de la j:

```
28    public static void muestraVectorForClasico(int[][] vector){
29        for(int i=0; i<vector.length; i++) {
30            System.out.println();
31            for(int j=0; j<vector[i].length; j++)
32                System.out.print("\t"+vector[i][j]);
33        }
34    }
35 }
```

(x)= Variables Breakpoints

Name	Value
vector	(id=16)
i	0
j	1

Como vemos ya la j a tomado el valor 1. Vamos a avanzar hasta casi el final del bucle para ver la situacion:

Nos situamos en la situacion que ahora esta en el bucle de la i en la posicion 1 y la j en 2:

```
28    public static void muestraVectorForClasico(int[][] vector){
29        for(int i=0; i<vector.length; i++) {
30            System.out.println();
31            for(int j=0; j<vector[i].length; j++)
32                System.out.print("\t"+vector[i][j]);
33        }
34    }
35 }
```

(x)= Variables Breakpoints

Name	Value
vector	(id=16)
i	1
j	2

```
For Clasico:

        0       12      15
        251     5
```

Ahora vamos a hacer lo mismo pero con un for mejorado:
Como anteriormente, pasamos a establecer el punto de ruptura en el for mejorado:

```
20    public static void muestraVectorForMejorado(int[][] vector){
21        for (int i[] : vector) {
22            System.out.println();
23            for(int x: i)
24                System.out.print("\t"+x);
25        }
26    }
```

Pasamos a ejecutarlo en modo depurador y lo recorremos con Step Over (F6).

```
20    public static void muestraVectorForMejorado(int[][] vector){
21        for (int i[] : vector) {
22            System.out.println();
23            for(int x: i)
24                System.out.print("\t"+x);
25        }
26    }
```

(x)= Variables | Breakpoints

Name	Value
vector	(id=16)
i	(id=19)
[0]	0
[1]	12
[2]	15

Como vemos ahora la variable i del bucle i toma valores que se encuentran en cada posicion del array.
Vamos a irlo recorriendo y observando su recorrido.

```
20    public static void muestraVectorForMejorado(int[][] vector){
21        for (int i[] : vector) {
22            System.out.println();
23            for(int x: i)
24                System.out.print("\t"+x);
25        }
26    }
```

(x)= Variables | Breakpoints

Name	Value
vector	(id=16)
i	(id=19)
[0]	0
[1]	12
[2]	15
x	0

Ahora vemos en el siguiente bucle ya la x nos toma el valor 0 que es lo que contiene el vector en esa posicion determinada.
En el siguiente salto ya nos imprime el valor en la pantalla.

```
LeyendoVectores [Java App
For Mejorado:

        0
```

Avancemos algunos saltos mas y observemos.

```
20    public static void muestraVectorForMejorado(int[][] vector){
21        for (int i[] : vector) {
22            System.out.println();
23            for(int x: i)
24                System.out.print("\t"+x);
25        }
26    }
```

(x)= Variables | Breakpoints

Name	Value
vector	(id=16)
i	(id=20)
[0]	251
[1]	5
[2]	2
x	2

Como vemos ya el bucle i a avanzado a la siguiente linea y contiene esos valores. Y la x toma el valor que le corresponde.
Un salto mas y vemos como imprime el numero 2.

```
For Mejorado:

        0       12      15
        251     5       2
```

Si dentro de cualquier bucle for pulsamos la tecla F7(Step Return) completa el bucle sin pasar paso a paso dentro de él y nos muestra el resultado final del bucle.